Cognitive Load from EEG + fNIRS — Complete Project Record

Everything about this project in one document: what it is, what state it was in, what was audited, what was built, every experiment that was run, what the numbers actually are, and what is still open.

All paths below are relative to the repository root; the document links are relative to this file.

Companion documents:

- `Cognitive_Load_Project_Report.pdf` — the illustrated 19-page report (figures included); regenerate with `.venv/bin/python -m research.make_pdf`
- `PROJECT_OVERVIEW.md` — the original research project
- `API_AND_ARCHITECTURE.md` — endpoint-by-endpoint API reference
- `BINARY_MODEL_IMPROVEMENT.md` — the experiment framework's methodology
- `RESULTS_BINARY_IMPROVEMENT.md` — machine-generated results (regenerate, never hand-edit)
- `RESULTS.md` — the original research results
- `NOVELTY_DANN.md` — the original domain-adversarial write-up

Part I — What the project is

1. The problem

Estimate **mental workload** from two brain signals recorded simultaneously:

- **EEG** — electrical activity, 200 Hz, fast, noisy, measures cortical rhythms.
- **fNIRS** — haemodynamic activity, 10 Hz, slow, measures blood oxygenation (HbO = oxygenated, HbR = deoxygenated haemoglobin).

They are complementary: EEG catches fast electrical shifts, fNIRS the slower metabolic response. Fusing them is the modelling premise.

2. The dataset

TU Berlin open dataset (Shin et al., 2018) — simultaneous EEG + NIRS during an n-back working-memory task.

Property	Value
Subjects	26
EEG	30 channels @ 200 Hz
fNIRS	36 channels @ 10 Hz (HbO and HbR each)
Trials	25 per subject (650 total)
Task	n-back working memory
Labels	0-back = **Low**, 2-back = **Medium**, 3-back = **High**

The label is not a subjective rating — it is the experimentally imposed difficulty, which is what makes it usable ground truth.

Binary framing (this work): keep `Cognitive_Load ∈ {0, 2}` — 0-back as **Low**, 3-back as **High** — and drop the 2-back middle class. Perfectly balanced: 2,288 windows per class.

3. Feature representation

Trials are sliced into overlapping windows. Each window becomes **1,476 features**:

```
EEG    540 = 30 electrodes × 18 feature types
             (6 bands: delta, theta, alpha, beta, gamma, engagement
              × 3 statistics: mean, std, slope)

fNIRS  936 = 36 optodes × 26 feature types
             (13 statistics: mean, std, slope, min, max, ptp, auc, power,
              skew, kurtosis, early, late, delta   × HbO and HbR)
```

Column names are **channel-major** — all of `ch00`'s features, then `ch01`'s, and so on. This matters later: it is what makes a 1D CNN legitimate.

Three windowed datasets exist, differing only in window geometry:

Key	File	Rows (binary)	Geometry	Adjacent overlap
`sliding_10s_5s`	`tuberlin_sliding.csv`	4,576	10 s window, 5 s stride	50%
`sliding_10s_1s`	`tuberlin_sliding_augmented.csv`	21,216	10 s window, 1 s stride	90%
`sliding_8s_4s`	`tuberlin_sliding_8s_4s.csv`	5,824	8 s window, 4 s stride	50%

Part II — State at the start of this session

The project already had a defensible pipeline: preprocessing, a hybrid model, 26-fold LOSO evaluation, DANN, SHAP, a FastAPI backend and a seven-page React app. Three things were missing for the task of *improving* the binary model without fooling ourselves:

1. **No separation between the data used to choose a model and the data used to report its score.** 26-fold LOSO is a good generalization estimate, but if you run thirty experiments and keep the best LOSO number, that number is a selection artifact — the folds have become a training signal.
2. **No run archive.** Artifacts went to a shared `outputs/` and were overwritten, so a result could not be traced to the code, config and data that produced it.
3. **No automated leakage assertions.** Leakage-freedom was a property of how the scripts happened to be written, not something checked before each fold.

Two corrections to the existing documentation

The data was not missing. `explanation/PROJECT_OVERVIEW.md` and `API_AND_ARCHITECTURE.md` both stated that `data/` was absent and that the 3-class research checkpoint could not be regenerated. **The data is present** — 9.3 GB under `ZDATASET/`, containing all three windowed feature sets, the full-trial set, and the raw `.mat` files. It is now referenced from `data/` by symlink. Both documents were corrected.

`config.py` and the docs disagreed on checkpoint selection. `config.py` sets `CHECKPOINT_METRIC = "val_loss"`; the documentation describes validation macro F1. The new framework uses **validation macro F1**, which is what a class-balance-sensitive task needs.

Part III — What was built

Everything is **additive**. `serving/`, `serving_3class/`, `backend/model_service.py` and every legacy training script are untouched, so the existing 2-class and 3-class demo behaves exactly as before.

4. The `research/` package

File	Lines	Responsibility
`research/runctx.py`	260	Run directories, status lifecycle, metadata, NaN-safe JSON, failure preservation
`research/datamod.py`	400	Dataset loading, the preprocessing contract, LOSO fold generation, per-fold scaler fitting
`research/leakage.py`	330	Every leakage assertion; raises `LeakageError` and aborts on any failure
`research/archs.py`	330	Model zoo — MLP and 1D-CNN encoders, concat/attention fusion, gradient reversal + domain discriminator
`research/train.py`	300	Training loop: early stopping, LR schedules, modality dropout, DANN, per-epoch eval-mode train metrics
`research/metrics.py`	290	Every metric, computed from predictions only
`research/plots.py`	230	Every figure, written into the run that produced it
`research/explain.py`	260	SHAP, t-SNE, subject-invariance probe
`research/experiment.py`	250	One spec evaluated across one fold set
`research/campaign.py`	330	The staged search, the development/lockbox split, the selection rule
`research/report.py`	380	Generates the results document from run artifacts
`research/verify_run.py`	150	Independently recomputes a run's metrics from its predictions
`research/test_leakage.py`	200	Negative tests — injects each leakage type, asserts the audit catches it
`research/test_framework.py`	160	Protocol correctness tests
`run_campaign.py`	620	CLI entry point; defines the staged experiment plan

5. Model architecture

All models share one interface so the trainer, explainability code and DANN path are architecture-agnostic:

```
forward(eeg, fnirs) -> logits
encode(eeg, fnirs)  -> fused embedding      (the DANN attack point)
gates(eeg, fnirs)   -> (eeg_gate, fnirs_gate) or None
attach_domain_head(n_domains)
forward_dann(eeg, fnirs, lambda_)
```

`DualStreamNet` — two modality encoders → fusion → classification head.

```
EEG   (540) ──► encoder ──┐
                          ├──► fusion ──► FC head ──► softmax
fNIRS (936) ──► encoder ──┘
```

Encoder family "mlp" (the served baseline):

```
Linear(n_features → proj_dim) → BatchNorm → ReLU → Dropout
  → N × ResMLPBlock(proj_dim)         [Linear→BN→ReLU→Dropout→Linear→BN, + skip]
  → Linear(proj_dim → enc_dim) → BatchNorm → ReLU
```

Encoder family "cnn" (added this session):

```
EEG   (B, 540) → reshape (B, 30 electrodes, 18 feats) → transpose → Conv1d(in=18)
fNIRS (B, 936) → HbO (B, 36, 13) ⊕ HbR (B, 36, 13) → (B, 36, 26) → Conv1d(in=26)
  → [ConvBlock | ResConvBlock] × len(filters)
  → concat(mean-pool, max-pool over channels)
  → Linear → BatchNorm → ReLU → enc_dim
```

Why the CNN is legitimate here. The columns are channel-major, so each modality block reshapes exactly onto a (channel, feature-type) grid. Convolving along the **electrode / optode axis**, with feature types as the convolution's input channels, shares one filter bank across all channels. That is a real inductive bias — look for the same spectral or haemodynamic pattern everywhere on the head — and it cuts the input-stage parameter count by more than an order of magnitude. The assumption it makes is that neighbouring channel indices are spatially related, which is only approximately true for this montage. That is exactly why it was run as a controlled experiment rather than adopted on recommendation.

Fusion:

- `ConcatFusion` — `Linear(2·in → out) → BN → ReLU → Dropout`
- `AttentionFusion` — a sigmoid gate per modality computed from both embeddings, each modality scaled by its own gate before fusing. The gate values are what the app's modality-contribution bar shows.

DANN: `GradientReversal` (identity forward, gradient × −λ backward) plus a `DomainDiscriminator` MLP on the fused embedding. Domain labels are the **training** subjects only, densely re-indexed per fold, so the discriminator never has a class for the held-out subject. λ follows the standard Ganin ramp.

Modality dropout (training only): each sample's EEG or fNIRS embedding is zeroed with configurable probability, never both at once. Gated by `self.training`, so validation, LOSO and lockbox always see both modalities.

6. Preprocessing contract

```
nan_to_num  →  [per-subject z-score]  →  log compression  →  scalers
```

- **EEG log**: band powers are non-negative and right-skewed → `log1p(clip(x, 0))`. When per-subject normalization runs *before* the log, values can go negative, so a signed log is used instead.
- **fNIRS log**: HbO/HbR go negative → signed `log1p`.
- **Scalers**: `RobustScaler` (EEG) + `StandardScaler` (fNIRS), refit from scratch on each fold's **training rows only**.

Per-subject normalization has three modes:

Mode	Statistics from	Deployment story
`transductive` (default)	all of the subject's windows	needs the whole session up front
`calibration`	the first *K* windows only	short calibration block, then live
`none`	—	nothing needed

Part IV — The evaluation protocol

7. The three-way subject split

The 26 subjects are partitioned once, deterministically from the seed:

```
development pool  18 subjects  [1,2,3,4,5,7,9,10,11,12,13,14,15,17,20,23,24,25]
lockbox            8 subjects  [6,8,16,18,19,21,22,26]
```

- **Experiment selection** runs 18-fold LOSO over the development pool. Every architecture, hyperparameter, scheduler, dropout rate and dataset variant is chosen on these folds alone.
- **The lockbox** is never read during the search. After the configuration is frozen, each lockbox subject is evaluated by a model trained only on the 18 development subjects. This is the estimate least contaminated by selection.
- **The 26-fold LOSO** is also reported, because it is the protocol the rest of the project uses. It is labelled *selection-informed* and read alongside the lockbox number, not instead of it.

8. Inside one fold

```
held-out subject   ──►  test only. Never in train, val, or any fitted statistic.
remaining subjects ──►  GroupShuffleSplit by subject  ──►  train + validation
```

Checkpoint selection uses **validation macro F1** with best-weight restore.

9. The selection rule — fixed in advance, applied mechanically

1. Rank arms by mean development-LOSO macro F1.
2. Take every arm within **one standard error** of the leader.
3. Inside that set prefer (a) the **incumbent**, then (b) the fewest parameters.

A challenger must clear the noise floor to be adopted.

> **A flaw I introduced and then caught.** The first implementation only broke ties on parameter count. Every arm in a stage often has *identical* parameter counts, so the rule silently degenerated into "pick the highest mean" — i.e. noise-chasing. It surfaced when stage S2 accepted a **+0.0023** improvement against a noise floor of **±0.018**. Over ten stages that compounds into a selection artifact. The rule was corrected to the one above and **the entire campaign was restarted**. The discarded run is still in the archive.

Noise floor. With 18 folds and a fold-to-fold standard deviation around 0.08, the standard error on mean macro F1 is **±0.018**. Differences smaller than that are reported but never acted on.

Part V — The leakage audit

`research/leakage.py` runs before every fold trains. Any failure raises `LeakageError` and aborts the run — a leaked score is never produced, so it can never be reported.

10. Per-fold checks (9)

Check	What it proves
`row_index_disjoint`	no row appears in two splits
`subject_disjoint`	train / val / test share no subject
`trial_disjoint`	no trial straddles a split
`window_identity_disjoint`	no `(subject, trial, window_id)` key on both sides
`duplicate_feature_rows`	no byte-identical feature row across splits
`scaler_fit_on_train_only`	rows used to fit the scalers ⊆ training rows
`heldout_subject_never_fitted`	the held-out subject is absent from train and val
`no_nan_or_inf`	preprocessing produced finite values
`labels_valid`	labels in range, training fold has both classes

11. Per-dataset checks (3)

Check	Result
`per_subject_norm_is_subject_local`	PASS — max abs delta **0.00e+00**
`no_duplicate_rows_in_dataset`	PASS — all 4,576 feature rows unique
`features_finite`	PASS

12. The sliding-window question

Windows overlap in time within a trial (50% at 5 s stride, 90% at 1 s stride). Overlapping windows share raw signal, so putting two of them on opposite sides of a split leaks.

Grouping by subject makes this structurally impossible: every window of the held-out subject is in the test set and none is anywhere else. The `window_identity_disjoint` and `trial_disjoint` checks assert it explicitly rather than relying on the argument.

13. Per-subject normalization — the honest caveat

Each subject's features are z-scored against that subject's own statistics, and this happens before the fold split.

It cannot leak between subjects. `assert_subject_local` proves this numerically rather than in prose: it re-runs the whole preprocessing chain for one subject *from that subject's raw rows in isolation* and checks the result matches the full-dataset computation bit-for-bit. If any cross-subject statistic had crept in, the isolated recomputation would differ. Observed difference: **exactly zero**.

It is transductive within a subject. Computing a subject's mean and standard deviation needs that subject's whole session. That is standard practice in BCI and uses no labels — but it means deployment requires a recorded session before the first prediction, not a single window. The `calibration` mode exists to measure that cost, and it was measured (see S10).

14. The audit is not vacuously true

`research/test_leakage.py` injects each leakage type deliberately and asserts the audit catches that specific check. **8/8 pass:**

Test	Result
clean fold passes all 9 checks	PASS
subject leakage caught	PASS
window leakage caught	PASS
scaler-fit leakage caught	PASS
duplicate-row leakage caught	PASS
non-finite values caught	PASS
held-out subject in val caught	PASS
per-subject norm is subject-local	PASS

`research/test_framework.py` asserts the protocol machinery. **6/6 pass**, including that early stopping restores weights which reproduce the best validation score to 1e-6.

Part VI — Every experiment

42 arms across 11 stages. Each stage varied one axis from the incumbent; the winner carried forward. All arms trained and evaluated on identical development folds.

15. Stage decisions

Stage	Axis	Best challenger	Δ vs incumbent	Noise floor	Outcome
`S1_batch_lr`	batch size, learning rate	`S1_bs64_lr1e3`	−0.0170	±0.0184	rejected
`S2_schedule`	epoch budget, LR schedule	`S2_no_sched`	+0.0022	±0.0184	rejected (inside noise)
`S3_weight_decay`	weight decay	`S3_wd0`	−0.0099	±0.0184	rejected
`S4_dropout`	dropout rates	`S4_drop_low`	+0.0085	±0.0184	rejected (inside noise)
`S5_capacity`	width, depth	`S5_proj192`	+0.0160	±0.0184	rejected (inside noise)
`S6_architecture`	MLP vs CNN, fusion	`S6_mlp_concat`	−0.0157	±0.0184	rejected
`S7_modality_dropout`	modality dropout	`S7_md010`	−0.0055	±0.0184	rejected
`S8_noise`	input noise, label smoothing	`S8_noise20`	−0.0072	±0.0184	rejected
`S9_dann`	domain-adversarial λ	`S9_dann01`	−0.0270	±0.0184	rejected
`S10_data`	window geometry, normalization, HbR	`S10_stride1s`	+0.0115	±0.0184	rejected (inside noise)

No stage produced a change that cleared the noise floor.

16. Full leaderboard — 18-fold development LOSO

Sorted best to worst. `Δ` and `p` are paired against the baseline on the same folds. `gap` is train − held-out macro F1 (higher = more overfitting).

Experiment	Stage	Macro F1	Accuracy	AUC	MCC	Δ vs base	p	gap	Params	Note
`S5_proj192`	S5	0.6611 ± 0.101	0.6686	0.721	0.348	+0.0160	0.309	0.146	620,570	projection width 192
`S10_stride1s`	S10	0.6566 ± 0.114	0.6620	0.721	0.333	+0.0115	0.461	0.217	245,594	dense windows: 10 s / 1 s stride
`S4_drop_low`	S4	0.6537 ± 0.120	0.6575	0.710	0.321	+0.0085	0.655	0.194	245,594	dropout 0.3 / 0.3
`S2_no_sched`	S2	0.6474 ± 0.094	0.6553	0.716	0.325	+0.0022	0.819	0.233	245,594	no LR schedule
`S5_proj32_enc32`	S5	0.6466 ± 0.112	0.6559	0.726	0.325	+0.0015	0.929	0.152	**66,114**	narrow: proj 32 / enc 32
`A0_baseline`	**S0**	**0.6451 ± 0.078**	**0.6537**	**0.717**	**0.324**	**baseline**	**—**	**0.163**	**245,594**	**served config — SELECTED**
`S5_blocks1`	S5	0.6433 ± 0.084	0.6474	0.704	0.302	−0.0018	0.890	0.158	207,578	1 residual block
`S4_drop_mid`	S4	0.6414 ± 0.104	0.6452	0.697	0.298	−0.0037	0.804	0.245	245,594	dropout 0.4 / 0.5
`S7_md010`	S7	0.6396 ± 0.100	0.6477	0.710	0.308	−0.0055	0.688	0.189	245,594	modality dropout 0.1 / 0.1
`S8_noise20`	S8	0.6380 ± 0.105	0.6474	0.714	0.306	−0.0072	0.635	0.184	245,594	input noise 0.20
`S3_wd0`	S3	0.6352 ± 0.110	0.6509	0.712	0.321	−0.0099	0.494	0.144	245,594	no weight decay
`S2_longer_p25`	S2	0.6316 ± 0.100	0.6420	0.702	0.299	−0.0136	0.107	0.214	245,594	120 epochs, patience 25
`S2_longer_p15`	S2	0.6316 ± 0.100	0.6420	0.702	0.299	−0.0136	0.107	0.214	245,594	120 epochs, patience 15
`S6_mlp_concat`	S6	0.6294 ± 0.101	0.6360	0.683	0.283	−0.0157	0.230	0.157	236,282	concat fusion (attention ablation)
`S2_cosine`	S2	0.6294 ± 0.113	0.6376	0.701	0.284	−0.0157	0.270	0.254	245,594	cosine annealing
`S3_wd01`	S3	0.6287 ± 0.107	0.6402	0.709	0.294	−0.0164	0.140	0.207	245,594	weight decay 0.1
`S3_wd003`	S3	0.6283 ± 0.098	0.6370	0.701	0.287	−0.0168	0.296	0.166	245,594	weight decay 0.003
`S1_bs64_lr1e3`	S1	0.6281 ± 0.122	0.6345	0.694	0.278	−0.0170	0.292	0.236	245,594	batch 64, lr 1e-3
`S8_noise10`	S8		0.6278	0.684	0.270	−0.0264	0.094	0.151	245,594	

Experiment	Stage	Macro F1	Accuracy	AUC	MCC	Δ vs base	p	gap	Params	Note
		0.6187 ± 0.097								input noise 0.10
S9_dann01	S9	0.6182 ± 0.098	0.6259	0.680	0.260	−0.0270	0.104	0.160	245,594	DANN λ 0.1
S10_win8s	S10	0.6160 ± 0.105	0.6223	0.677	0.255	−0.0291	0.128	0.172	245,594	8 s windows / 4 s stride
S7_md_eeg_only	S7	0.6150 ± 0.091	0.6247	0.676	0.259	−0.0301	0.172	0.175	245,594	drop EEG more often (0.3 / 0.1)
S10_norm_after_log	S10	0.6148 ± 0.099	0.6237	0.685	0.260	−0.0303	**0.050**	0.173	245,594	per-subject norm after the log
S7_md020	S7	0.6110 ± 0.117	0.6275	0.697	0.264	−0.0341	0.209	0.157	245,594	modality dropout 0.2 / 0.2
S5_proj48	S5	0.6061 ± 0.106	0.6105	0.654	0.228	−0.0390	0.144	0.118	113,402	projection width 48
S10_calibration_norm	S10	0.6049 ± 0.115	0.6241	0.672	0.263	−0.0402	0.156	0.185	245,594	norm from 30 calibration windows
S8_smooth	S8	0.6048 ± 0.141	0.6259	0.678	0.262	−0.0403	0.171	0.183	245,594	label smoothing 0.1
S1_bs64	S1	0.5978 ± 0.120	0.6158	0.677	0.252	−0.0473	**0.019**	0.115	245,594	batch 64
S9_dann05	S9	0.5968 ± 0.117	0.6127	0.669	0.237	−0.0483	**0.014**	0.156	245,594	DANN λ 0.5
S1_bs128	S1	0.5864 ± 0.104	0.6067	0.688	0.234	−0.0587	**0.009**	0.113	245,594	batch 128
S3_wd005	S3	0.5846 ± 0.116	0.6007	0.661	0.209	−0.0605	**0.004**	0.222	245,594	weight decay 0.05
S5_blocks3	S5	0.5841 ± 0.100	0.6057	0.652	0.235	−0.0610	**0.011**	0.135	283,610	3 residual blocks
S8_noise0	S8	0.5831 ± 0.094	0.6013	0.652	0.217	−0.0620	**0.025**	0.134	245,594	no input noise
S10_no_hbr	S10	0.5781 ± 0.098	0.5915	0.635	0.191	−0.0670	**0.014**	0.086	200,666	HbO only (drop HbR)
S9_dann10	S9	0.5778 ± 0.110	0.5979	0.663	0.213	−0.0674	**0.012**	0.125	245,594	DANN λ 1.0
S10_no_subject_norm	S10	0.5698 ± 0.106	0.5887	0.635	0.190	−0.0753	**0.014**	0.125	245,594	

Experiment	Stage	Macro F1	Accuracy	AUC	MCC	Δ vs base	p	gap	Params	Note
										no per-subject normalization
`S6_cnn_res`	S6	0.5550 ± 0.085	0.5758	0.604	0.163	−0.0902	**0.002**	**0.026**	107,962	CNN + residual, concat
`S1_lr1e4`	S1	0.5547 ± 0.100	0.5900	0.646	0.208	−0.0904	**0.004**	0.063	245,594	lr 1e-4
`S6_cnn_res_attn`	S6	0.5467 ± 0.079	0.5597	0.588	0.125	−0.0984	**0.001**	**0.035**	117,274	CNN + residual + attention
`S6_cnn_wide`	S6	0.5467 ± 0.071	0.5660	0.606	0.142	−0.0984	**<0.001**	0.076	363,930	CNN + residual + attention, 128 filters
`S4_drop_high`	S4	0.5083 ± 0.127	0.5650	0.602	0.150	−0.1369	**<0.001**	**0.040**	245,594	dropout 0.6 / 0.7
`S6_cnn`	S6	0.4786 ± 0.087	0.5312	0.552	0.075	−0.1665	**<0.001**	**0.026**	54,842	1D CNN, concat fusion

Part VII — Final results

17. The selected model

`A0_baseline` — the configuration already being served. 245,594 parameters.

```
{
  "encoder": "mlp", "proj_dim": 96, "enc_dim": 48, "n_blocks": 2,
  "encoder_dropout": 0.5, "fusion": "attention",
  "fc_units": [48, 24], "fc_dropout": 0.6,
  "optimizer": "adamw", "lr": 0.0003, "weight_decay": 0.01,
  "batch_size": 16, "max_epochs": 35,
  "scheduler": "plateau", "lr_patience": 4, "lr_factor": 0.5,
  "early_stop_patience": 7, "augment_noise": 0.05,
  "class_weights": "balanced", "label_smoothing": 0.0,
  "modality_dropout_eeg": 0.0, "modality_dropout_fnirs": 0.0,
  "use_dann": false
}
```

Data: `sliding_10s_5s`, per-subject normalization `transductive` (`before_log`), HbR retained.

18. Headline numbers

26-fold LOSO (all 26 subjects) — mean ± std across folds

Metric	Value
Accuracy	**0.6309 ± 0.1011**
Macro F1	**0.6255 ± 0.1044**
ROC-AUC	0.6775 ± 0.1289
MCC	0.2675 ± 0.2058
Cohen's Kappa	0.2618 ± 0.2022

Lockbox — 8 subjects never used for any selection decision

Metric	Value
Accuracy	**0.6371 ± 0.0850**
Macro F1	**0.6275 ± 0.0931**
ROC-AUC	0.6888 ± 0.1112
MCC	0.2816 ± 0.1690

> The lockbox number is **not lower** than the LOSO number. That is the check the whole protocol existed for: 42 experiments of searching produced **zero selection inflation**.

19. Train / validation / test accuracy

Protocol	Train	Validation	Test (held-out subject)	Gap
26-fold LOSO	0.8022 ± 0.093	0.6400 ± 0.053	**0.6309 ± 0.101**	+0.171
Lockbox	0.8186 ± 0.059	0.6783 ± 0.038	**0.6371 ± 0.085**	+0.182

"Testing" means a **person the model has never seen**. There is no random train/test split anywhere in this work. A random window split would score much higher, but it would be measuring the model's ability to recognise a subject's physiological baseline, not workload.

20. Pooled confusion matrix (26-fold LOSO, 4,576 held-out windows)

	predicted Low	predicted High
true Low	1,561	727
true High	962	1,326

Class	Precision	Recall	F1	Support
Low	0.619	0.682	0.649	2,288
High	0.646	0.580	0.611	2,288

Pooled accuracy 0.6309 · macro F1 0.6299 · ROC-AUC 0.6707 · MCC 0.2632 · Kappa 0.2618. Chance = 0.50.

21. Per-subject spread (26-fold LOSO, macro F1)

```
S17 0.81  S3 0.80  S7 0.76  S25 0.75  S16 0.72  S10 0.70  S9 0.69
S22 0.67  S11 0.67  S24 0.67  S1 0.67  S15 0.67  S6 0.65  S18 0.65
S13 0.64  S20 0.63  S23 0.60  S8 0.57  S4 0.56  S26 0.52  S21 0.50
S2  0.49  S19 0.48  S5 0.48  S14 0.47  S12 0.43
```

Best S17 (0.807), worst S12 (0.431). **4 of 26 subjects fall below chance accuracy.** The spread across people (0.38) is more than twice the largest difference between any two models tested (0.17). That is the finding: the dominant source of error is *which person you are testing on*, not which model you chose.

Part VIII — What the experiments found

22. The CNN does not help this representation

Architecture	Macro F1	Δ	p	gap	Params
MLP baseline	0.6451	—	—	0.163	245,594
CNN + residual	0.5550	−0.090	0.002	**0.026**	107,962
CNN + residual + attention	0.5467	−0.098	0.001	**0.035**	117,274
CNN wide (128 filters)	0.5467	−0.098	<0.001	0.076	363,930
CNN plain	0.4786	−0.167	<0.001	**0.026**	54,842

Every variant is significantly worse. The interesting column is `gap`: the CNNs cut overfitting from 0.163 to 0.026–0.076 — electrode-axis weight sharing nearly **eliminates** it. But it removes so much capacity that held-out performance falls further than the overfitting it cures.

Interpretation: on **pre-aggregated per-channel statistics** — not raw time series — the spatial weight-sharing prior costs more than it buys. A CNN would be the right tool on the raw signal; on 18 summary numbers per electrode there is no local structure along the channel axis for it to exploit.

23. Attention fusion is mildly supported

Replacing attention with plain concatenation (`S6_mlp_concat`) loses 0.016 macro F1 — the right direction, but not significant ($p = 0.230$). Attention is retained on the basis that it is not worse and it produces the interpretable modality gates the app displays.

24. Regularization cannot close the gap

Weight decay (0 → 0.1), dropout (0.3 → 0.7), input noise (0 → 0.2), label smoothing and modality dropout (0.1 → 0.3) were all tested. **None improved held-out performance beyond noise.**

The pattern is consistent and one-directional: strong regularization reliably reduces the gap *and* reduces held-out performance with it. `S4_drop_high` (dropout 0.6/0.7) cuts the gap from 0.163 to 0.040 while losing 0.137 macro F1 ($p < 0.001$).

The gap is not excess capacity that can be regularized away. It reflects genuine between-subject variability — the model fits its training subjects well and those subjects are simply not fully representative of the next person.

25. Capacity is not the constraint in either direction

`S5_proj32_enc32` reaches 0.6466 with **66,114 parameters** versus the baseline's 245,594 — statistically identical (Δ +0.0015, $p = 0.929$) at **3.7× smaller**. Widening to 620,570 parameters (`S5_proj192`) gains +0.016, inside the noise floor.

Practical note: if model size matters for your deployment, the 66K model is free. It was not adopted only because the selection rule prefers the incumbent on a statistical tie.

26. DANN is rejected — and the probe explains why

λ	Macro F1	Δ	p
0 (baseline)	0.6451	—	—
0.1	0.6182	−0.027	0.104
0.5	0.5968	−0.048	0.014
1.0	0.5778	−0.067	0.012

Monotonic degradation with adversarial strength — independently replicating the project's earlier 3-class finding, now on the binary task under a stricter protocol.

The subject-invariance probe explains the mechanism. A fresh classifier trained to recover subject identity from each representation:

Representation	Probe accuracy	Chance
raw features	0.013	0.125
baseline model embedding	0.214	0.125
selected model embedding	0.214	0.125
DANN model embedding	**0.217**	0.125

Per-subject z-scoring has already stripped subject identity from the *raw* features (far below chance). The **encoder then re-introduces it** (0.214), and **DANN fails to remove it** (0.217 — no better than the baseline it was meant to improve). So the adversary injects noise into the task head without deleting the confound it targets. That is a coherent negative result, not an inconclusive one.

27. S10 confirmed three existing design choices — and priced a fourth

Variant	Δ macro F1	p	Reading
No per-subject normalization	**−0.0753**	0.014	it genuinely matters
Drop HbR (HbO only)	**−0.0670**	0.014	HbR genuinely helps
Normalize *after* the log	**−0.0303**	0.050	`before_log` was the right call
Calibration-only norm (30 windows)	−0.0402	0.156	~0.04 cost, not significant
Dense windows (1 s stride, 4.6× rows)	+0.0115	0.461	inside noise — no free lunch
8 s windows / 4 s stride	−0.0291	0.128	10 s is better

The **calibration-only** result is the practically useful one: restricting per-subject statistics to the first 30 windows — a short calibration block rather than the whole session — costs about 0.04 macro F1 and is **not statistically significant**. That makes an online deployment defensible.

The **dense-window** result matters too: `config.py` defaults to the 1 s-stride file, whose adjacent windows overlap 90%. It inflates the row count 4.6× and buys nothing measurable.

28. The training schedule was already near-optimal

Batch 64/128 and lr 1e-4 are all significantly worse ($p \le 0.019$). Longer budgets, cosine annealing and removing the LR schedule all land inside noise. The short 35-epoch budget with plateau scheduling is itself acting as regularization — the mean best epoch is 14.5, so early stopping is doing real work.

29. Feature importance (SHAP, `GradientExplainer`)

Computed on this run's own trained model over held-out windows.

Modality	Share of total |SHAP|
fNIRS	**0.945**
EEG	0.055
— HbO	0.458
— HbR	0.488

Top feature families:

Rank	Family	Modality	Total mean |SHAP|
1	HbR auc	fNIRS	0.3016
2	HbO max	fNIRS	0.2824
3	HbO min	fNIRS	0.2716
4	HbO mean	fNIRS	0.2695
5	HbR min	fNIRS	0.2592
6	HbR power	fNIRS	0.2504
7	HbO late	fNIRS	0.2443
8	HbR max	fNIRS	0.2412

Top individual features: `hbo_ch23_min`, `hbr_ch24_max`, `hbr_ch28_min`, `hbo_ch24_power`, `hbo_ch23_max`, `hbr_ch06_skew`, `hbr_ch13_auc`, `hbr_ch28_mean`.

Two things follow. First, **fNIRS dominates overwhelmingly** — far more starkly than the original project's near-parity attention gates suggested. Second, **HbR slightly exceeds HbO** (0.488 vs 0.458), which independently corroborates the S10 ablation showing HbR removal costs 0.067 macro F1. The dominant signal is haemodynamic response *shape* — auc, max, min, mean, late — not EEG band power.

Part IX — The run archive

30. Layout

A `RunContext` is created **before any data is read**, so a run that crashes during loading still leaves a directory with its metadata, config and error. Run ids embed a timestamp plus a random suffix and directory creation uses `exist_ok=False`: a run can never overwrite another.

```
outputs/runs/run_<YYYYmmdd>_<HHMMSS>_<hex4>/
├── run_metadata.json      environment, git commit, dataset, seed
├── config.json            the exact resolved configuration
├── dataset_info.json      provenance, schema, class semantics, preprocessing
├── run_status.json        created | running | completed | failed  (+ stage, warnings)
├── run_summary.json/.md   final summary
├── leakage_audit.json     dataset-level audit result
├── metrics/               metrics.json/.csv, per_class, loso, per_subject
├── predictions/           predictions.csv, probabilities.csv
├── confusion_matrix/      json + png
├── curves/                training/validation curves, ROC, PR (json + png)
├── feature_importance/    shap_values.json, feature_ranking.csv, pngs
├── representations/       t-SNE coordinates (csv + json), embeddings, pngs
├── model/                 model.pt, per-fold checkpoints, scaler, schema, labels
├── experiments/           every arm's record, leaderboard.csv, stage_log.json
├── folds/
└── logs/                  training.log, error.log
```

31. Predictions are the source of truth

`predictions/predictions.csv` stores one row per evaluated window: experiment id, fold, subject, trial, window id, window start time, true label, predicted label, full probability vector, confidence and correctness. **Every metric in `metrics.json` — and therefore every number on the Evaluation page — is computed from that file.**

`research/verify_run.py` proves it: it reads *only* `predictions.csv`, recomputes the entire metric record from scratch, and diffs it against what the run wrote.

```
$ .venv/bin/python -m research.verify_run --all
  OK  run_20260819_175542_dfa8: 1232 predictions, 19 scalar metrics recomputed
  OK  run_20260819_175652_d013: 1232 predictions, 19 scalar metrics recomputed
  OK  run_20260819_181550_c044: 2200 predictions, 20 scalar metrics recomputed
  OK  run_20260820_141126_19cb: 4576 predictions, 19 scalar metrics recomputed
  4 verified, 0 mismatched
```

32. The deployment model carries no performance claim

Each run also trains one model on **all** subjects and saves it under `model/`. It has no held-out data, so its `model_config.json` says explicitly that it carries no performance claim. Reported metrics come from the per-fold models under `model/folds/`.

33. Runs in the archive

Run	Status	Contents
`run_20260820_141126_19cb`	completed	**The final run.** S9, S10, 26-fold LOSO, lockbox, SHAP, t-SNE
`run_20260819_184443_f0d1`	interrupted	S1–S8, 33 arms. Stopped by operator; all arms intact
`run_20260819_181550_c044`	completed	3-class verification (reduced cohort — warning recorded)
`run_20260819_175820_33bd`	interrupted	The discarded run with the flawed selection rule
`run_20260819_175652_d013`	completed	Smoke test (reduced cohort — warning recorded)
`run_20260819_175542_dfa8`	**failed**	Preserved with stage, exception and traceback
`run_20260819_174414_4f9e`	interrupted	Early smoke test

Total 111 MB. **Nothing was deleted, including the failures.**

34. Determinism across runs

The campaign spans two runs because it was interrupted and resumed. Merging their arms is legitimate because the baseline arm was re-run in the second run and **reproduced the first bit-for-bit**:

```
run 1 dev-LOSO macro F1  0.645125
run 2 dev-LOSO macro F1  0.645125     identical
run 1 pooled accuracy    0.653725
run 2 pooled accuracy    0.653725     identical
```

Part X — Backend and frontend

35. New API surface

```
GET /api/runs                          run history (filters: mode, status)
GET /api/runs/{id}                     metadata, config, dataset, audit, manifest
GET /api/runs/{id}/metrics             metrics + explanations + LOSO + per-class
GET /api/runs/{id}/confusion-matrix
GET /api/runs/{id}/curves              ROC + PR points
GET /api/runs/{id}/training-history    per-fold and fold-averaged curves
GET /api/runs/{id}/per-subject
GET /api/runs/{id}/feature-importance  SHAP
GET /api/runs/{id}/representations     t-SNE + subject-invariance probe
GET /api/runs/{id}/experiments         leaderboard + stage log
GET /api/runs/{id}/experiments/{exp}   one arm's full record
GET /api/runs/{id}/predictions         paged raw predictions
GET /api/runs/{id}/summary
GET /api/runs/{id}/report              run_summary.md as text
GET /api/runs/{id}/figure?path=...     a PNG the run rendered (path-guarded)
```

Served by `backend/runs_service.py`. Nothing is computed in the request path, so the API cannot disagree with the artifacts. Path traversal is blocked (400) and missing artifacts 404 with a message naming the expected file.

`stale` is computed, not stored: a run that recorded itself as `running` but has not updated in 30 minutes had its process killed. The status on disk is left untouched.

36. New pages

`/evaluation` — Evaluation Metrics. A run selector controls the whole page; selecting a run reloads every section from that run alone, so run A's metrics can never appear beside run B's plots. The run id is mirrored into the URL (`/evaluation?run=<id>`) so a view is linkable. Thirteen sections:

1. Model, protocol and dataset
2. Held-out performance (with lockbox, generalization gap, paired tests vs baseline)
3. Classification metrics (overall, macro/weighted/micro, probability quality)
4. Per-class metrics (precision, recall, F1, specificity, support, TP/FP/FN/TN)
5. Confusion matrix
6. ROC analysis
7. Precision-Recall analysis
8. Per-subject held-out performance (chart + table)
9. Training vs validation curves (loss, macro F1, accuracy, macro recall)
10. Feature importance (modality shares, top features, top families)
11. Learned representation (t-SNE, switchable variant and colouring; subject probe)

12. Experiment comparison (chart, stage log, full leaderboard)
13. Run details (provenance, leakage audit, configuration)

`/runs` — **Run History.** Every run with status, dataset, protocol, metrics and warnings; filterable by mode and status; each row opens that run's evaluation. Failed runs show their failure stage and message.

37. Nothing is hardcoded

Class names, class count, chance level and metric validity all come from the selected run's own artifacts. There is no `["Low","Medium","High"]` constant and no `if binary` branch anywhere in the new pages.

Metric validity is decided by the data. Top-K is emitted only when `K < n_classes`; one-vs-one AUC only when `n_classes > 2`. Anything undefined is reported as `null` with a stated reason, which the UI renders instead of a number:

```
"unavailable": {
  "roc_auc_macro_ovo": "one-vs-one is undefined for binary classification (identical to OVR)",
  "top_3_accuracy": "Not applicable: K=3 is not smaller than the number of classes (2)."
}
```

Verified in a browser for a binary run, a 3-class run, a completed run, a running run and a failed run.

Part XI — Limitations

1. **Statistical power.** With 18 development subjects, the standard error on mean macro F1 is ±0.018, so this search could only reliably detect improvements larger than roughly **0.04**. A smaller genuine gain would be invisible to it.
2. **26 subjects is small** for cross-subject claims, and 4 of them fall below chance. The between-subject spread (0.38 macro F1) dominates everything else measured here.
3. **Per-subject normalization is transductive.** It uses no labels and no other subject's data, but it needs the subject's whole session. The `calibration` mode prices that at about 0.04 macro F1 (not significant), which is the number to quote for an online deployment.
4. **The greedy staged search is not exhaustive.** It varies one axis at a time from a moving incumbent, so it cannot find improvements that require two changes simultaneously.
5. **The 26-fold LOSO number is selection-informed** — the configuration was chosen using 18 of the 26 subjects it evaluates on. The lockbox number is the clean estimate, and it agrees.
6. **SHAP is one attribution method on one trained model.** The fNIRS dominance (0.945) is much starker than the original project's attention-gate analysis suggested; the S10 HbR ablation corroborates the direction but not the magnitude.
7. **The CNN result is specific to this feature representation.** It says a CNN over the channel axis of pre-aggregated statistics does not help. It says nothing about a CNN over the raw time series, which was never tested.
8. **`xgboost` is unusable in this environment** (missing `libomp`), so the classical sweep was not re-run under the new protocol.

Part XII — Reproducing

```
# Environment
brew install git-lfs && git lfs install --local && git lfs pull

# Full campaign (several hours on CPU)
.venv/bin/python run_campaign.py --tag my-run

# A single stage
.venv/bin/python run_campaign.py --stages S6

# 3-class instead of binary
.venv/bin/python run_campaign.py --mode 3class

# Smoke test: reduced cohort and epoch budget, every code path in ~2 minutes
.venv/bin/python run_campaign.py --quick --limit-subjects 7 --n-lockbox 3 --max-epochs 6

# Tests
.venv/bin/python -m research.test_leakage      # 8/8
.venv/bin/python -m research.test_framework    # 6/6
.venv/bin/python -m research.verify_run --all  # recompute metrics from predictions

# Regenerate the results document from artifacts (never hand-edit it)
.venv/bin/python -m research.report outputs/runs/<final_run> \
    --also-arms-from outputs/runs/<earlier_run> -o RESULTS_BINARY_IMPROVEMENT.md

# The app
cd backend && ../.venv/bin/python -m uvicorn main:app --port 8000
cd frontend && npm run dev            # http://localhost:5173
```

`--limit-subjects` and `--max-epochs` record a warning into the run, and that warning is displayed on the Evaluation page, because a reduced-cohort run is not a valid performance estimate.

Part XIII — Conclusion

The binary model was not improved, and that is the result.

Across **42 controlled experiments** spanning batch size, learning rate, LR schedule, epoch budget, weight decay, dropout, capacity, depth, architecture family (MLP vs 1D CNN), fusion type, modality dropout, input noise, label smoothing, domain-adversarial training, window geometry, normalization mode and HbR inclusion — **nothing beat the served configuration by more than the noise floor**, and most alternatives were significantly worse.

The configuration the project had already locked in sits at a local optimum for this feature representation. The evidence that the remaining error is *not* a modelling deficiency is threefold and mutually consistent:

1. **Regularization trades the gap for performance one-for-one.** Every setting that shrank the train–test gap shrank held-out accuracy with it.
2. **The subject probe shows the encoder re-introduces subject identity that the preprocessing had already removed, and DANN cannot remove it again.**
3. **The between-subject spread (0.38 macro F1) is more than twice the largest difference between any two of the 42 models tested (0.17).**

The final honest number is **63% accuracy on people the model has never seen** (chance 50%), and the lockbox confirms it is not inflated by the search.

The most promising untested direction is not a better classifier. It is either more subjects, or a genuinely different feature representation — the raw time series rather than 18 summary statistics per electrode — where an architectural prior like convolution would have something to work with.